Campus Venue Booking System

DBMS Course Project — Database Design Document (Mid-Sem) — Lab: P1, Group-2

Team: Areesha Shahid (2023B1A40991H), Harshitha Ambavarapu (2023B2AD0873H),
Rudhra Koul (2023B3AD0445H), Geetanjali Dangeti Sree (2023B4AD0609H), Pratyush Pranay (2023B5AA1211H)

1. Requirements

Problem. Campus events (pro shows, stand-up, movie screenings, fest events, talks) are held in the auditorium and in classrooms, but tickets are sold without seat numbers, with no protection against over-selling and no data for organizers. The system lets students, faculty and guests book tickets online — exact seats in the auditorium, general-admission places in classrooms — with subsidised student pricing, and gives organizing clubs occupancy and revenue reports.

Users. *Attendees* (Student, Faculty, Guest): browse events, view the seat map, hold and book seats, pay, view or cancel bookings. *Club managers*: create events and shows, set the price grid, cancel shows, view dashboards. Venues and seats are set up by the database administrator through scripts.

Key functionality. Event/show search and filters; interactive seat map; 5-minute seat holds during checkout; bookings, tickets and payments; cancellations; organizer reports (occupancy, revenue by event and attendee type, top events).

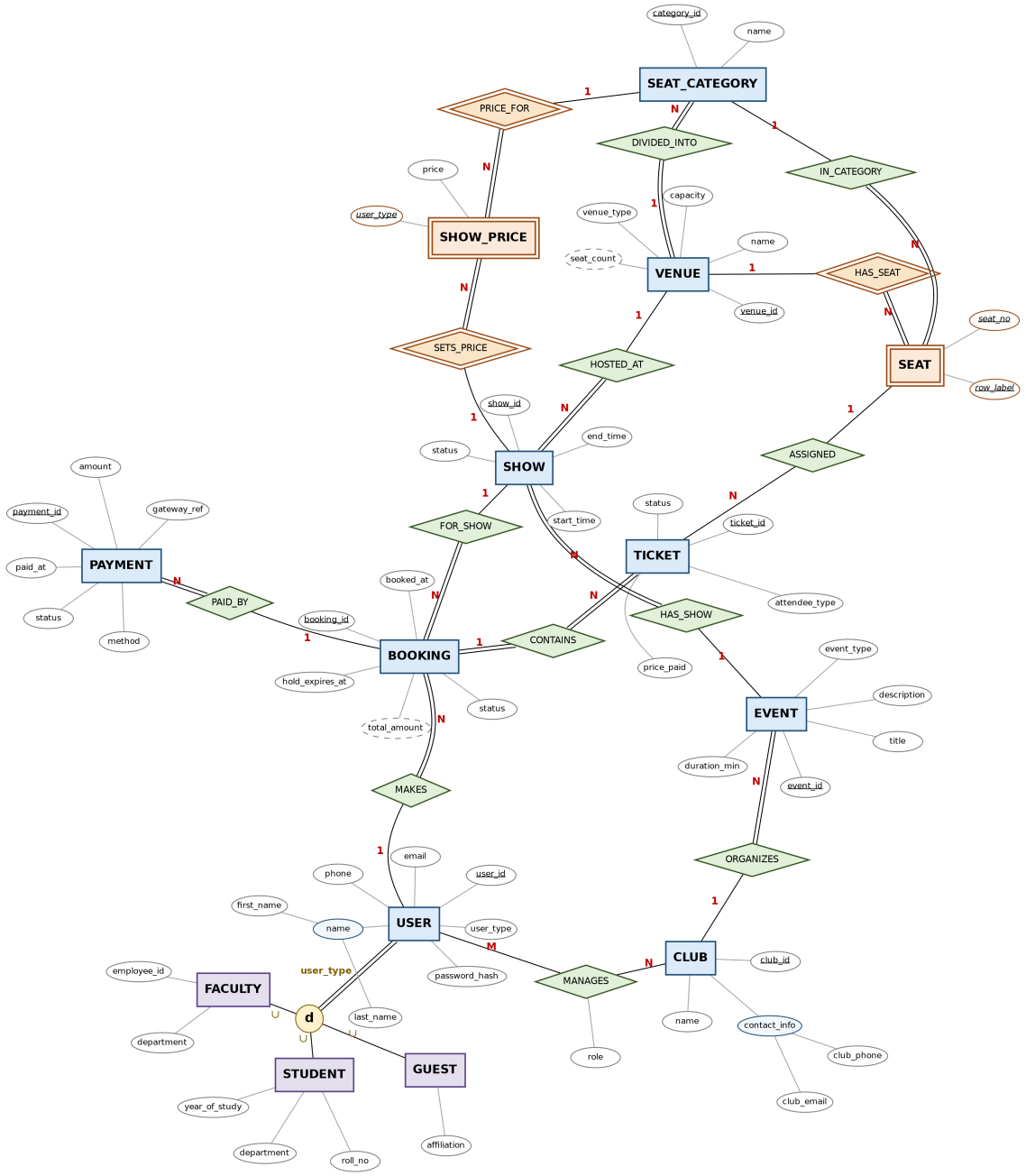
2. Assumptions

- A1. Single campus. Venues are AUDITORIUM (numbered seats) or CLASSROOM (large shared desks, so general admission up to a fixed capacity).
- A2. Seat categories (pricing zones) are assigned per seat, not per row. Each classroom has exactly one category, “General”.
- A3. Every user is exactly one of Student, Faculty or Guest. Student and faculty tickets are subsidised; guests pay full price.
- A4. Lecture timetables are outside the system: organizers book classrooms only for slots that are free of classes.
- A5. Payments go through a simulated gateway; the system records each attempt.
- A6. last_name is optional (single-name users); each user has one contact phone.

3. Business rules and how they are enforced

ID	Rule	Enforced by
BR1	A seat can have at most one live (HELD or ACTIVE) ticket per show — no double booking.	Partial UNIQUE index on ticket
BR2	Two non-cancelled shows cannot overlap in time in the same venue.	EXCLUDE constraint on show
BR3	Classroom tickets have no seat and live tickets ≤ capacity; auditorium tickets must have a seat; seats exist only in auditoriums.	CHECK + triggers
BR4	A ticket’s seat belongs to the venue of the ticket’s show; a seat’s category belongs to the seat’s venue.	Composite FKs
BR5	Price is set per (show, seat category, attendee type) and frozen on the ticket as price_paid.	PK on show_price + trigger
BR6	A booking is for exactly one show and holds at most 6 live tickets.	Composite FK + trigger
BR7	Bookings are allowed only for SCHEDULED shows that have not started.	Trigger
BR8	A hold lasts 5 minutes; an unpaid booking then becomes EXPIRED and its HELD tickets free their seats.	hold_expires_at + procedure + trigger
BR9	Each user has one type; the subclass row must match it and the type cannot change once a subclass row exists. User and subclass row are created together.	PK = FK + triggers; registration transaction
BR10	Cancelling a booking cancels its tickets and refunds its payment; cancelling a show cancels all its bookings.	Triggers
BR11	Only managers of a club may create or edit that club’s events, shows and prices.	club_manager + backend check
BR12	A show’s price grid may only use seat categories of the show’s venue.	Trigger
BR13	A show must last at least as long as its event’s duration.	Trigger
BR14	A subsidised ticket must match the buyer’s own type, max one per person per show; extra tickets are GUEST-priced.	Trigger
BR15	A booking has at most one successful payment.	Partial UNIQUE index on payment

4. EER diagram (Chen notation)

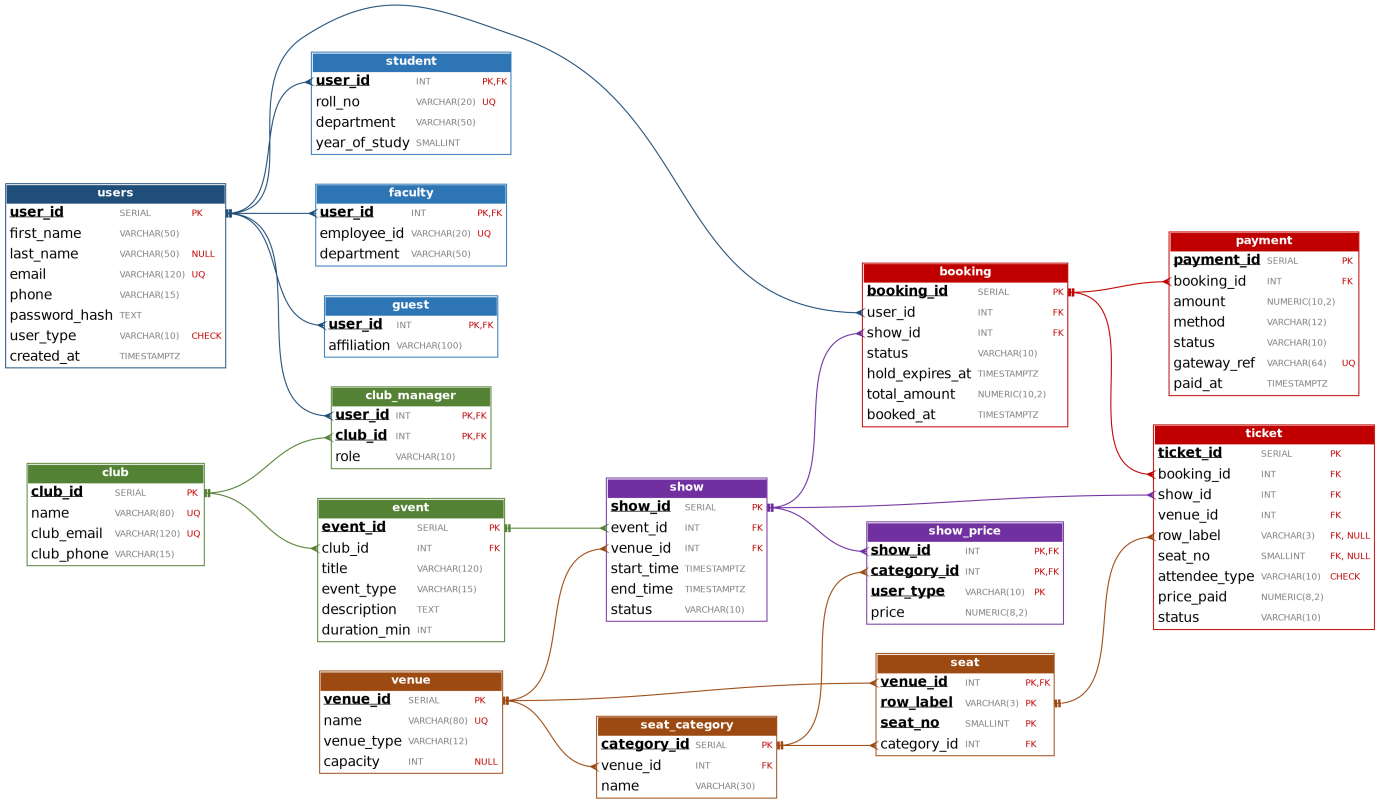


Notation: rectangle = strong entity; double rectangle = weak entity; diamond = relationship; double diamond = identifying relationship; underlined = key; *italic underlined* = partial key; ellipse with sub-ellipses = composite; dashed ellipse = derived; double line = total participation; red 1/N/M = cardinality; circle "d" = disjoint specialization, double line to it = total, ∪ = subset.

Key modelling decisions.

- **SEAT** is weak: a seat cannot exist without its venue, and (row_label, seat_no) is unique only within one venue, so its key borrows venue_id via HAS_SEAT. VENUE's side is partial because classrooms have no seats.
- **SHOW_PRICE** is a weak entity with two owners, SHOW (SETS_PRICE) and SEAT_CATEGORY (PRICE_FOR): a price means nothing without both. Its partial key user_type tells apart the prices within one show and category, so its full key is (show, category, user_type).
- **TICKET.attendee_type** records who the ticket is for, which may differ from the buyer (a student can buy guest tickets for friends). The ticket's price is looked up with it, and it enforces the one-subsidised-ticket rule (BR14).
- **Specialization** of USER into STUDENT, FACULTY, GUEST is disjoint, total and attribute-defined on user_type (labelled on the diagram); each subclass has attributes the others lack.
- **A seat hold is a TICKET in state HELD** inside a PENDING BOOKING with hold_expires_at, so CONTAINS is total on both sides and no separate hold entity is needed.
- **Total participation on the "one" side** (every VENUE has a category; every BOOKING has a ticket; every USER has a subclass row) cannot be enforced by foreign keys. It is guaranteed by creating parent and children in one transaction (venue setup script, seat hold, registration).
- **Derived:** BOOKING.total_amount (sum of live ticket prices) and VENUE.seat_count (auditorium capacity). **Composite:** USER.name, CLUB.contact_info.

5. Relational schema



Bold underlined = primary key; crow's foot = referencing (many) side. Composite FKs are drawn once: ticket → booking (booking_id, show_id); ticket → show (show_id, venue_id); ticket → seat (venue_id, row_label, seat_no); seat → seat_category (category_id, venue_id). Seat columns of a ticket are NULL for classroom tickets.

ER/EER → relational mapping

Rule	Applied to	Result
Strong entity → table	USER, CLUB, VENUE, SEAT_CATEGORY, EVENT, SHOW, BOOKING, TICKET, PAYMENT	9 tables; composite attributes flattened (first_name, last_name; club_email, club_phone)
Weak entity → table, PK = owner key(s) + partial key	SEAT; SHOW_PRICE (two owners)	seat(venue_id, row_label, seat_no); show_price(show_id, category_id, user_type, price)
1:N → FK on N side (NOT NULL if total)	ORGANIZES, HAS_SHOW, HOSTED_AT, DIVIDED_INTO, IN_CATEGORY, MAKES, FOR_SHOW, CONTAINS, ASSIGNED, PAID_BY	e.g. event.club_id, show.venue_id, ticket.booking_id; ASSIGNED FK is nullable (partial)
M:N → new table	MANAGES	club_manager(user_id, club_id, role)
Specialization → superclass + subclass tables (option 8A)	STUDENT, FACULTY, GUEST	PK of each subclass is also FK → users ON DELETE CASCADE
Derived attributes	seat_count, total_amount	seat_count not stored (view v_venue_capacity); total_amount stored, kept in sync by trigger

15 relations: 11 from entities (incl. weak SEAT and SHOW_PRICE), 3 from the specialization and 1 from the M:N relationship. The four non-entity tables are required by the mapping rules rather than added for size.

6. Integrity constraints (enforced by the DBMS)

Relation	Keys	NOT NULL / UNIQUE / CHECK / other
users	PK user_id; UNIQUE email	first_name, email, password_hash, user_type NOT NULL; user_type CHECK IN (STUDENT, FACULTY, GUEST); email UNIQUE, CHECK LIKE '%_@_%'; phone CHECK pattern; last_name nullable
student / faculty / guest	PK = FK user_id → users CASCADE	roll_no, employee_id NOT NULL UNIQUE; department NOT NULL; year_of_study CHECK 1–5; trigger: user's user_type must match; user_type locked once a subclass row exists (BR9)
club	PK club_id	name, club_email NOT NULL UNIQUE; club_phone CHECK pattern
club_manager	PK (user_id, club_id); FKs → users, club CASCADE	role CHECK IN (ADMIN, MEMBER)
venue	PK venue_id	name UNIQUE; venue_type CHECK IN (AUDITORIUM, CLASSROOM); capacity > 0; CHECK ((venue_type = CLASSROOM) = (capacity IS NOT NULL))
seat_category	PK category_id; FK venue_id → venue CASCADE	UNIQUE (venue_id, name); UNIQUE (category_id, venue_id) as FK target; trigger: one category per classroom
seat	PK (venue_id, row_label, seat_no); FK (category_id, venue_id) → seat_category	row_label CHECK ^[A-Z]{1,3}$; seat_no > 0; trigger: auditorium only (BR3)
event	PK event_id; FK club_id → club	title NOT NULL; event_type CHECK IN (…); duration_min > 0
show	PK show_id; FKs event_id, venue_id; UNIQUE (show_id, venue_id)	CHECK end_time > start_time; status CHECK IN (SCHEDULED, CANCELLED, COMPLETED); **EXCLUDE USING gist (venue_id WITH =, tstzrange(start_time, end_time, '[)') WITH &&) WHERE status <> CANCELLED** (BR2); trigger BR13
show_price	PK (show_id, category_id, user_type); FKs → show, seat_category (both CASCADE)	user_type CHECK IN (…); price >= 0; trigger BR12
booking	PK booking_id; FKs user_id, show_id; UNIQUE (booking_id, show_id)	status CHECK IN (PENDING, CONFIRMED, CANCELLED, EXPIRED); total_amount >= 0; CHECK hold_expires_at > booked_at; trigger BR7
ticket	PK ticket_id; FK (booking_id, show_id) → booking; FK (show_id, venue_id) → show; FK (venue_id, row_label, seat_no) → seat	CHECK ((row_label IS NULL) = (seat_no IS NULL)); attendee_type CHECK IN (…); price_paid >= 0; status CHECK IN (HELD, ACTIVE, CANCELLED, EXPIRED); **UNIQUE INDEX (show_id, venue_id, row_label, seat_no) WHERE status IN (HELD, ACTIVE)** (BR1); trigger: BR3, BR5, BR6, BR14
payment	PK payment_id; FK booking_id CASCADE; UNIQUE gateway_ref	amount > 0; method, status CHECK IN (…); CHECK (status IN (SUCCESS, REFUNDED)) = (paid_at IS NOT NULL); UNIQUE INDEX (booking_id) WHERE status = SUCCESS (BR15)

Why these constraints

- **Double booking is impossible at the database level.** Two concurrent holds on the same seat both try to insert a HELD ticket; the partial unique index lets exactly one commit. Cancelled and expired tickets remain for history but no longer block the seat.
- **Classrooms** have no seat to collide on, so the ticket trigger locks the show row (SELECT … FOR UPDATE) before checking capacity, serializing concurrent buyers.
- **Composite foreign keys** make ticket.show_id always equal its booking's show and ticket.venue_id always equal its show's venue, so the seat FK cannot point into another building.
- **The venue-clash rule** uses a half-open range [start, end), so back-to-back shows are allowed; cancelled shows free their slot.
- **Not enforced by constraints:** "at least one child" rules (see Section 4) are guaranteed by transactions; club-manager permissions (BR11) are checked by the backend.
- **Referential actions:** CASCADE only where the child cannot exist without the parent (subclass rows, seats, categories, prices, tickets, payments); RESTRICT elsewhere, so events, shows or users with bookings cannot be deleted accidentally.

7. Design justification

The main schema-design decisions, the alternatives considered, and why each choice was made.

- **Removed redundancy found during review.** (a) VENUE originally stored both venue_type and seating_mode, but the type already decides the seating (auditorium = numbered, classroom = general admission), so seating_mode was dropped. (b) Stored capacity for auditoriums duplicated the number of seats and could go out of sync, so capacity is now stored only for classrooms (enforced by a CHECK) and computed for auditoriums in the view v_venue_capacity.
- **ticket stores show_id and venue_id even though they can be found through booking and show.** The no-double-booking rule (BR1) must be a database constraint, and a unique index can only use columns of its own table. Composite foreign keys make it impossible for these copies to disagree with booking and show. The alternative (checking double booking in a trigger) is weaker when two users book at the same moment. This is also why the schema has ticket.show_id although the ER has no TICKET–SHOW relationship: it is an implementation choice, not a new fact.
- **A hold is a HELD ticket, not a separate entity.** One table and one unique index cover both holding and selling, instead of two tables that must be checked against each other. A later Redis/TTL cache can sit in front of the holds without weakening BR1, because the database index remains the final check.
- **Classroom tickets use NULL seat columns** instead of splitting TICKET into seated and general-admission subclasses: one table keeps BR1 a single index and keeps booking queries simple. The NULL pair is controlled by a CHECK and the venue-type trigger.
- **Surrogate category_id instead of the natural key (venue_id, name).** With the natural key, show_price would have to store venue_id, which the show already determines, so the same fact would be stored twice. The surrogate key also makes renaming a zone a one-row change.
- **SEAT keeps a natural composite key.** A seat is identified by (venue, row, number) in real life, never renamed, and a surrogate id would still need the same UNIQUE constraint.
- **user_type is a checked attribute, not a separate entity.** It is the defining attribute of the specialization and appears in users, show_price and ticket (as attendee_type) with the same CHECK on the three fixed values, so the values can never disagree. A separate type entity was considered and rejected: it would express a user’s type twice (entity link and subclass) and adding a type would need a new subclass anyway.
- **Price is a weak entity with two owners.** A price exists only for a specific show and seat category; user_type distinguishes the prices within that pair. Making it an attribute of SHOW or SEAT_CATEGORY alone would be impossible, because it depends on both.
- **Stored derived total_amount and snapshot price_paid.** total_amount is needed while a booking is PENDING (it is the amount sent to the payment gateway) and is recomputed by a trigger on every ticket change. price_paid is a historical fact: later price changes must not alter past receipts.
- **Specialization mapped as superclass + one table per subclass** rather than one wide table, to avoid many NULL columns and to allow NOT NULL on subclass-specific attributes such as roll_no.
- **Surrogate keys for main entities** (user_id, show_id, booking_id …) because natural candidates such as email can change; natural candidate keys are still declared UNIQUE.
- **Status columns instead of deleting rows.** Cancelled and expired bookings and tickets are kept for reports and history; partial unique indexes make sure only live rows block seats.